

Knapsack Problem

Wykonane przez studentów DSW: Jakub Buzuk dsw49445 i Maksym Belilovskyi
49438

W dokumencie ReadMe mamy instrukcje do odpalenia programu. Po instalowaniu wszystkich niezbędnych moduły Pythonu oraz stworzenia wirtualnego otoczenia za pomocą komendy:

```
python3 -m venv .venv
source .venv/bin/activate
```

Możemy uruchomić skrypt dla działania programu:

```
python -m knapsack_cli --population_size=100 --generations=100 --
mutation_rate=0.001 --input_file_name=low-dimensional/f10_1-
d kp 20 879
```

```
Lord@LAPTOP-LKTC6CJ6 MINGW64 /d/Dsw_genetic_algorithms_and_neural_networks/assignment_1_knapsack_problem (development)
$ python -m knapsack_cli --population_size=100 --generations=100 --mutation_rate=0.001 --input_file_name=low-dimensional/f10_l-d_kp_20_879
Solving knapsack problem with 20 items and a capacity of 879
Using settings:
Population size: 100
Generations: 100
Crossover type: single_point
Mutation rate: 0.001
Selection type: roulette_wheel
Fitness type: linear

100%|██████████████████████████████████████| 100/100 [00:00<00:00, 163.49it/s]
Best solution found: Knapsack(total_weight=828, total_value=990, capacity=879)
```

Jak możemy zobaczyć, program pokazuje najlepsze rozwiązanie bliskie do optimum.

Jeżeli chcemy zmienić jakieś parametry przy odpaleniu programy, to należy skorzystać:

```
python -m knapsack cli -help
```

Dodamy do naszego kodu parametr `--enable_plots true`, żeby mieć możliwość zobaczyć grafik, opisujący poszukiwanie najlepszego rozwiązania:

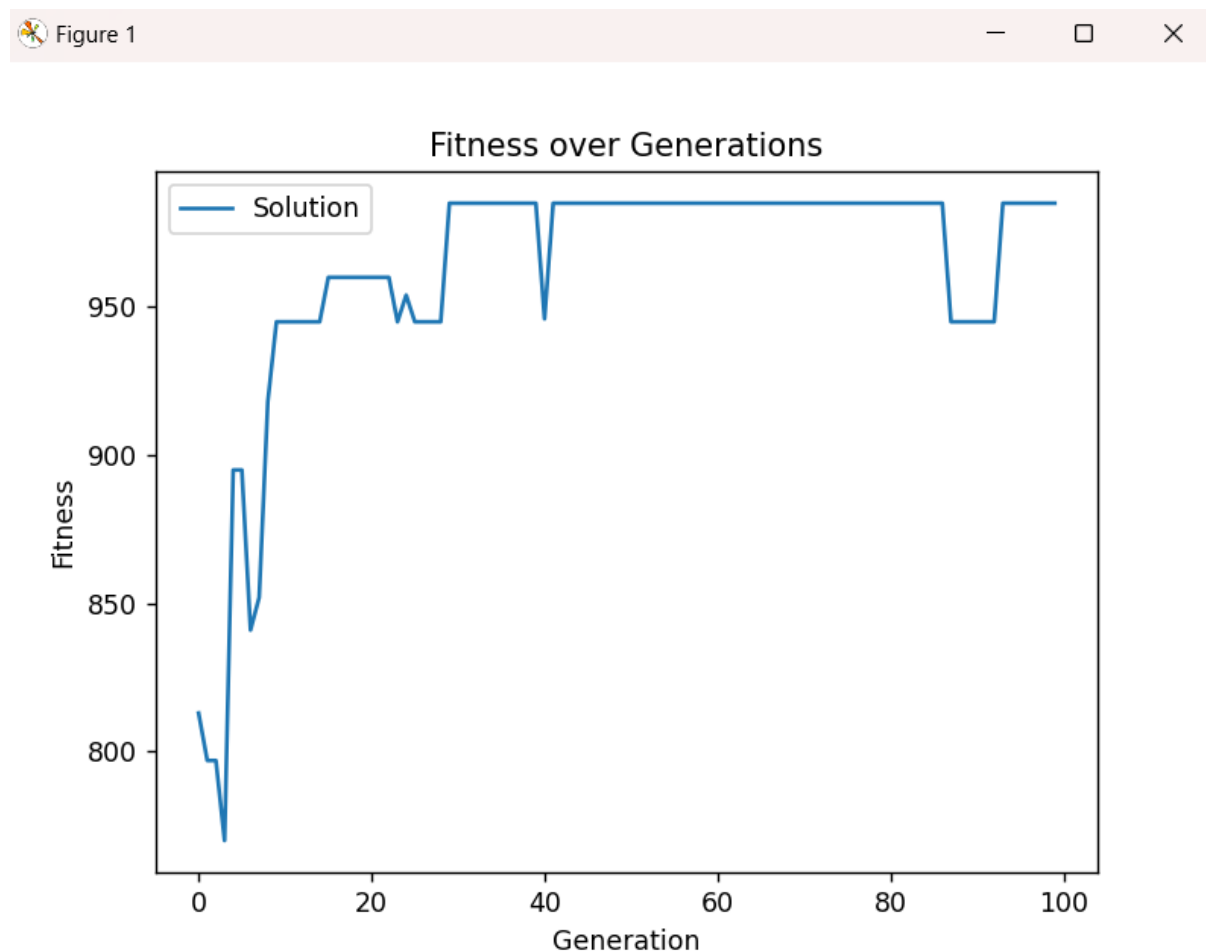
```

lord@LAPTOP-LKTC6CJ6 MINGW64 /d/DSW_genetic_algorithms_and_neural_networks/assign
ment_1_knapsack_problem (development)
$ python -m knapsack_cli --population_size=100 --generations=100 --mutation_rate
=0.001 --input_file_name=low-dimensional/f10_1-d_kp_20_879 --enable_plots true
Solving knapsack problem with 20 items and a capacity of 879
Using settings:
Population size: 100
Generations: 100
Crossover type: single_point
Mutation rate: 0.001
Selection type: roulette_wheel
Fitness type: linear

100%|████████████████████████████████████████| 100/100 [00:00<00:00, 161.60it/s]
Best solution found: Knapsack(total_weight=842, total_value=985, capacity=879)
(.venv)

```

Za chwilę zobaczymy grafik



Na grafiku możemy zobaczyć proces znalezienia najlepszego rozwiązania przez algorytm.

Możemy również włączyć opcje pokazywania grafików dla wszystkich sposobów selekcji (rankingowa, selekcyjna etc,) za pomocą komendy `--compare_solvers true`

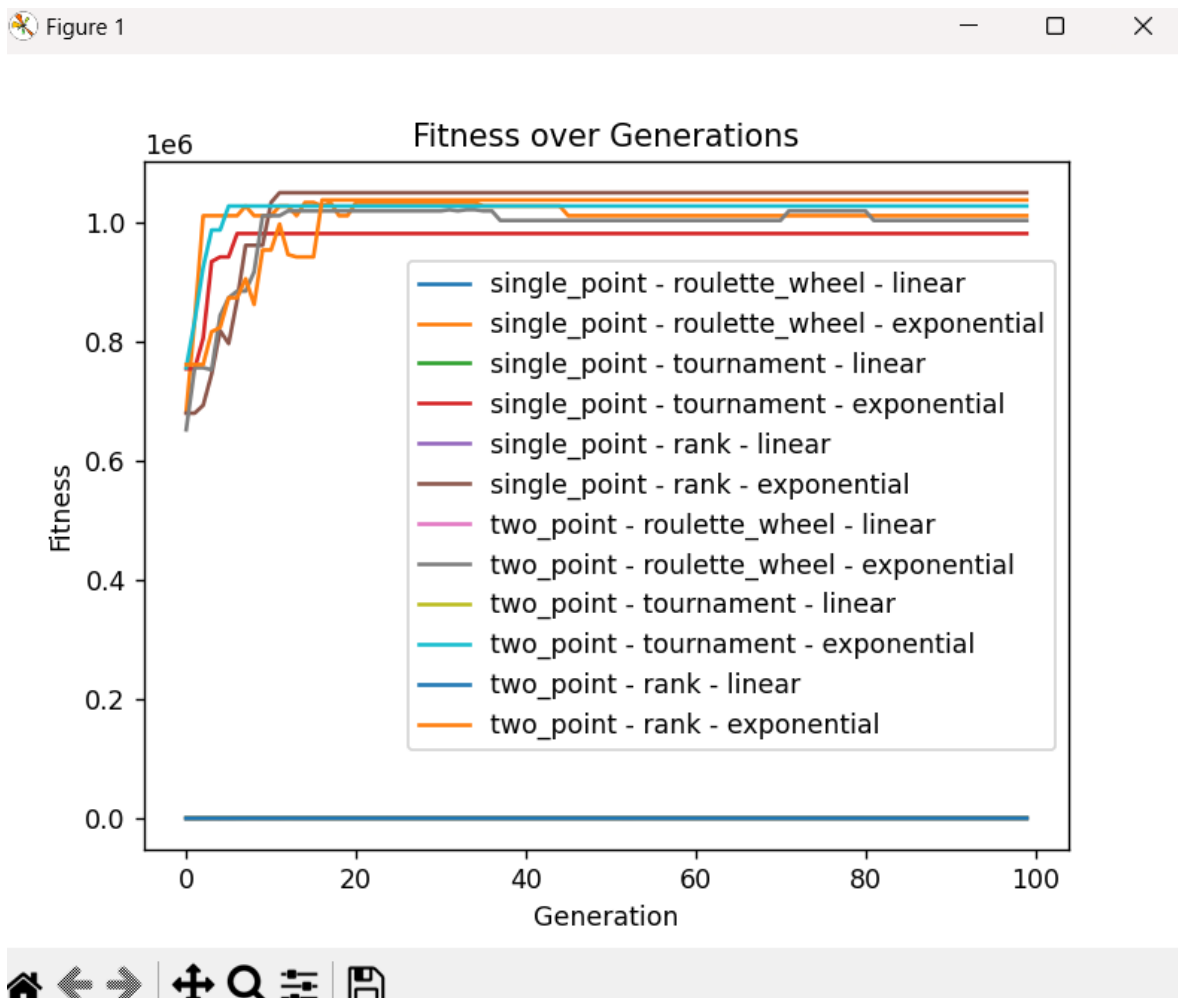
```

$ python -m knapsack_cli --population_size=100 --generations=100 --mutation_rate=0.001 --input_file_name=low-dimensional/f10_l-d_kp_20_879 --enable_plots true --compare_solvers true
Solving knapsack problem with 20 items and a capacity of 879
Using settings:
Population size: 100
Generations: 100
Comparing solvers with different settings:
100% | 100/100 [00:00:00:00, 151.051t/s]
100% | 100/100 [00:00:00:00, 144.451t/s]
100% | 100/100 [00:00:00:00, 115.321t/s]
100% | 100/100 [00:00:00:00, 113.951t/s]
100% | 100/100 [00:00:00:00, 122.971t/s]
100% | 100/100 [00:00:00:00, 154.041t/s]
100% | 100/100 [00:00:00:00, 124.211t/s]
100% | 100/100 [00:00:00:00, 122.301t/s]
100% | 100/100 [00:01:00:00, 94.761t/s]
100% | 100/100 [00:01:00:00, 94.011t/s]
100% | 100/100 [00:00:00:00, 119.101t/s]
100% | 100/100 [00:00:00:00, 118.231t/s]

Best solutions found:
single_point - roulette_wheel - linear: Knapsack(total_weight=850, total_value=984, capacity=879)
single_point - roulette_wheel - exponential: Knapsack(total_weight=845, total_value=1006, capacity=879)
single_point - tournament - linear: Knapsack(total_weight=872, total_value=1005, capacity=879)
single_point - tournament - exponential: Knapsack(total_weight=861, total_value=991, capacity=879)
single_point - rank - linear: Knapsack(total_weight=871, total_value=1025, capacity=879)
single_point - rank - exponential: Knapsack(total_weight=871, total_value=1025, capacity=879)
two_point - roulette_wheel - linear: Knapsack(total_weight=867, total_value=950, capacity=879)
two_point - roulette_wheel - exponential: Knapsack(total_weight=821, total_value=1002, capacity=879)
two_point - tournament - linear: Knapsack(total_weight=837, total_value=1019, capacity=879)
two_point - tournament - exponential: Knapsack(total_weight=851, total_value=1014, capacity=879)
two_point - rank - linear: Knapsack(total_weight=871, total_value=1025, capacity=879)
two_point - rank - exponential: Knapsack(total_weight=837, total_value=1019, capacity=879)
(cenv)

```

Za chwilę dostaniemy grafik:



Jak można zobaczyć, każdy sposób selekcji się kończy rozwiązaniem, które jest bliskim do optymalnego dla problemu plecakowego.